

int64 (integer 64 bits)

- **Ce que c'est** : Nombres entiers (sans virgule)
- **Exemples** : 1 , 42 , -5 , 1000
- **Concrètement** : Pour les âges, quantités, années, compteurs
- **Opérations possible** : Calculs mathématiques (+, -, ×, ÷), comparaisons

python

```
# Exemple de colonne int64  
ages = [25, 32, 28, 41] # → type int64
```

str (string / chaîne de caractères)

- **Ce que c'est** : Texte
- **Exemples** : "Jean", "Paris", "email@exemple.com", "123" (même les chiffres entre guillemets !)
- **Concrètement** : Noms, adresses, codes postaux (comme texte), catégories
- **Attention** : "123" en str n'est PAS un nombre, vous ne pouvez pas faire de calculs dessus

python

 Copy  Download

```
# Exemple de colonne str  
noms = ["Jean", "Marie", "Pierre"] # → type str ou object
```

object

- **Ce que c'est** : Type "fourre-tout" de pandas, souvent utilisé pour du texte
- **Concrètement** : Dans la pratique, `object` = texte (mélange de `str` et autres types)
- **Pourquoi existe** : Héritage historique. Pandas moderne utilise `string` au lieu de `object`

python

 Copy

 Downl

```
# Colonne avec type object (souvent du texte)
ville = ["Paris", "Lyon", "Marseille"] # → souvent object, parfois string
```

float64 (flottant 64 bits)

- Ce que c'est : Nombres décimaux (avec virgule)
- Exemples : 3.14 , 2.5 , 0.001 , 99.99
- Concrètement : Prix, pourcentages, mesures, moyennes

datetime64 (date/heure)

- Ce que c'est : Dates et heures
- Exemples : 2024-01-15 , 2023-12-25 14:30:00
- Concrètement : Permet des opérations comme "ajouter 7 jours", calculer des écarts

bool (booléen)

- Ce que c'est : Vrai ou Faux
- Exemples : `True` , `False`
- Concrètement : Indicateurs oui/non, cases à cocher

catégorie (category)

- Ce que c'est : Valeurs répétées limitées
- Exemples : `"M"` , `"F"` ; `"Bas"` , `"Moyen"` , `"Haut"`
- **Avantage** : Économise de la mémoire (stocke les codes une seule fois)

Pourquoi c'est important ?

python

Exemple concret

`df['age'] = [25, 30, 35]` *# int64 → calcul possible*

`df['age_str'] = ['25', '30', '35']` *# object/str → PAS de calcul direct !*

Ceci fonctionne :

`df['age'] + 5` *# → [30, 35, 40] ✓*

Ceci ne fonctionne PAS :

`df['age_str'] + 5` *# ✗ TypeError: can only concatenate str*

Pour corriger, il faut convertir :

`df['age_str'] = df['age_str'].astype('int64')` *# conversion str → int*

Comment vérifier les types ?

python

Afficher les types de toutes les colonnes

```
print(df.dtypes)
```

Pour une colonne spécifique

```
print(df['nom_colonne'].dtype)
```

Résumé complet (types + valeurs manquantes + mémoire)

```
print(df.info())
```

Problème fréquent : colonne object contenant des chiffres

python

 Copy  Down

```
# Si df.info() montre "object" pour une colonne de nombres  
# C'est souvent car il y a des espaces, des lettres, ou du texte mélangé  
  
# Exemple : df['prix'] = ["10 €", "20 €", "30 €"] # object ❌  
  
# Solution : nettoyer d'abord  
df['prix'] = df['prix'].str.replace(' €', '') # enlever "€"  
df['prix'] = df['prix'].astype('float64') # convertir en nombre
```

En résumé : Le type détermine **ce que vous pouvez faire** avec vos données. Un nombre en `int64/float64` se calcule, du texte en `object/str` se nettoie et se manipule différemment.